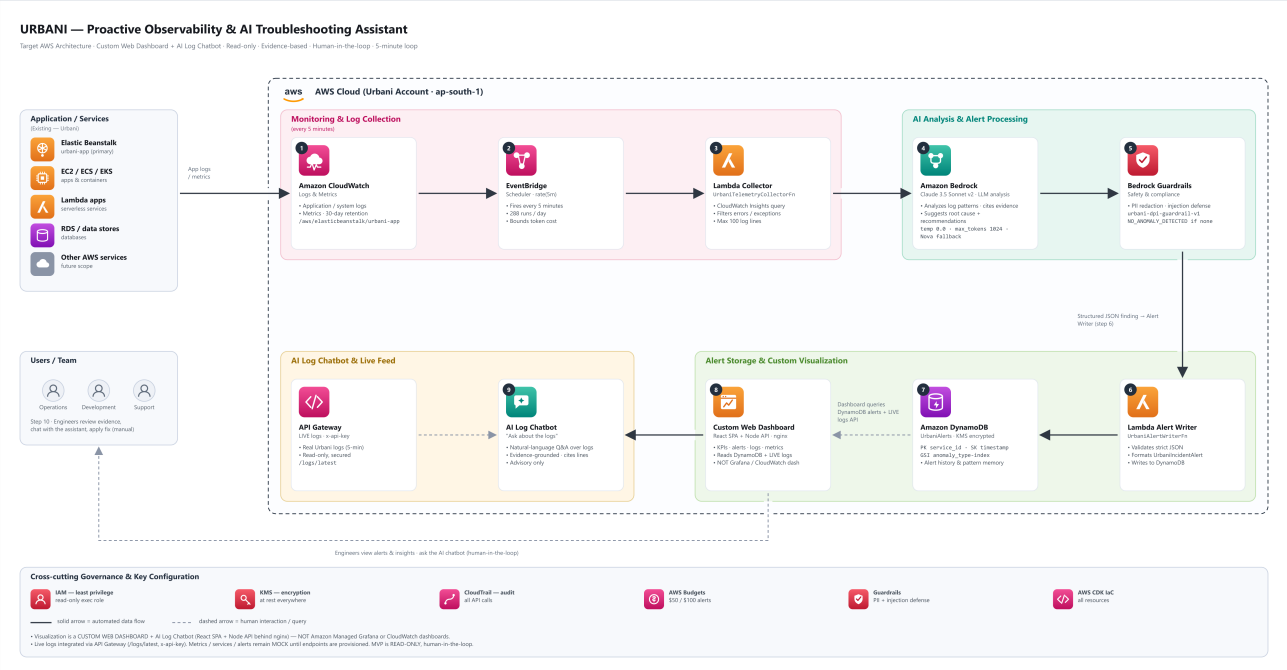
URBANI

Proactive Observability & AI Troubleshooting Assistant

Project Documentation

Complete System & Deployment Documentation

This document describes the delivered Urbani observability platform: its architecture, features, live AWS data integration, Google sign-in, roles and audit, automatic notifications, service health, export, and the ECS Fargate deployment, together with an honest status of what is verified and what is still pending. All content reflects the implemented system.



Target AWS architecture: ECS Fargate dashboard + serverless incident pipeline.

Table of Contents

1. **Executive Overview** 3
2. **System Architecture** 3
   - 2.1 Request flow 3
   - 2.2 Components 3
   - 2.3 Background services inside the backend 4
3. **Application Features** 4
   - 3.1 Pages and navigation 4
   - 3.2 Urbani Copilot (AI assistant) 5
   - 3.3 Log search and export 5
   - 3.4 Service health and uptime 6
   - 3.5 Dashboard personalisation 6
   - 3.6 Branding and dark mode 6
4. **Live Data Integration** 6
   - 4.1 Log evidence correlation 6
   - 4.2 Incident confidence 7
5. **Access Control and Security** 7
   - 5.1 Sign-in methods 7
   - 5.2 Google single sign-on 7
   - 5.3 Multi-factor authentication 7
   - 5.4 Roles and permissions 7
   - 5.5 User management 8
   - 5.6 Audit log 8
   - 5.7 Secrets and data protection 8
   - 5.8 Honest provenance model 8
6. **Automatic Alert Notifications** 9
   - 6.1 How it works 9
   - 6.2 Channels 9
   - 6.3 E-mail recipients by role 9
   - 6.4 Safe rollout and delivery log 9
7. **Deployment: AWS ECS Fargate** 10
   - 7.1 Runtime topology 10
   - 7.2 Deploy sequence 10
   - 7.3 Health checks 10
   - 7.4 Configuration reference 10
8. **Operations and Maintenance** 11
9. **Quality and Testing** 11
10. **Delivery Status and Open Items** 12
    - 10.1 Status by capability 12
    - 10.2 Before real users are added 13
    - 10.3 Inputs requested from Urbani 13
11. **Roadmap and Next Steps** 13
12. **Glossary** 14

1. Executive Overview

The Urbani Proactive Observability & AI Troubleshooting Assistant is a custom web platform that gives operations teams a single, honest view of their service health. It surfaces live logs and AI-generated incident alerts from the Urbani QA environment and provides an in-context AI assistant (the "Urbani Copilot") that answers questions grounded in the real log and incident evidence. It is advisory and human-in-the-loop: the platform never performs automated remediation.

The delivered platform includes:

- **Dashboard, alerts and logs:** health summary, alerts/incidents list and detail, advanced log search, metrics, services, usage and cost, and settings views.
- **Live AWS data:** real logs and AI-generated incidents from the Urbani QA API Gateway, with the Copilot answering through Amazon Bedrock (Nova 2 Lite) server-side.
- **Secure access:** password sign-in and Google single sign-on, optional authenticator-app MFA, four roles with a permission matrix, user management and an append-only audit log.
- **Automatic notifications:** alert rules that send e-mail (Google Workspace SMTP or Amazon SNS), Slack and Microsoft Teams messages, with a safe dry-run mode and a delivery log.
- **Operations tooling:** service health and uptime panel, log and alert export (CSV, JSON, PDF), Copilot history and feedback, a personalised dashboard, corporate branding with dark mode, and load-test tooling.
- **Honest provenance:** every data surface is labelled LIVE, MOCK or WAITING_FOR_INTEGRATION so viewers always know the source of what they see.
- **Production deployment:** containerized frontend and backend on ECS Fargate behind an Application Load Balancer with TLS, served at a dedicated domain.

> **How to read the status of each capability**
>
> Every capability in this document is built and covered by automated tests. A few depend on AWS access or on inputs only Urbani can provide, and have not yet been exercised against the real service. Section 10 lists, per capability, what is verified, what is pending, and what Urbani is asked to supply.

2. System Architecture

The application is a TypeScript monorepo with three packages: a shared types library, an Express backend, and a React (Vite) single-page frontend. In production the two runnable components run as one ECS Fargate task: an nginx container serves the SPA and reverse-proxies API calls to the backend container over the shared task network.

2.1 Request flow

The browser only ever talks to the frontend origin. nginx serves the SPA at "/" and proxies "/api/*" to the backend on port 4000; the backend host is never exposed directly. The backend reads live data through a strict adapter boundary, so the data source can change without any change to the API contract, the dashboard, or business logic.

2.2 Components

Component	Technology	Role
Frontend	React 18 + Vite, served by nginx	SPA dashboard; reverse-proxies /api to the backend. The only browser-facing component (port 8080).

Component	Technology	Role
Backend	Node.js 20 + Express	REST API, authentication and authorisation, live-data adapters, notification engine, health prober, persistence (port 4000).
Shared	TypeScript library	Common contracts, roles and permissions used by both backend and frontend.
Persistence	SQLite (default) or DynamoDB	Users, preferences, audit, chat history, alerts, notification rules and log, health results. The driver is chosen by one setting; both pass the same contract tests.
Identity	Google (OpenID Connect)	Optional single sign-on. Users authenticate at Google; roles are assigned inside the platform.
Incident pipeline	AWS CDK (EventBridge, Lambda, Bedrock, DynamoDB)	Serverless pipeline that generates incidents upstream (infrastructure-as-code).

2.3 Background services inside the backend

Service	What it does	Control
Notification engine	Polls alerts on a schedule, matches rules, de-duplicates and sends e-mail, Slack and Teams messages.	NOTIFICATIONS_ENABLED, ALERT_POLL_MINUTES (5)
Health prober	Probes configured health endpoints once a minute and stores UP / DEGRADED / DOWN results.	HEALTH_PROBES_ENABLED, HEALTH_PROBE_INTERVAL_SECONDS (60)
Retention jobs	Daily purge of audit events, chat history, notification log and health results past their retention period.	AUDIT_RETENTION_DAYS, CHAT_RETENTION_DAYS and others

3. Application Features

3.1 Pages and navigation

Navigation items and buttons are shown according to the signed-in role; the API enforces the same rules.

Page	Purpose
Dashboard	Service health strip, key metrics, system health, AI cost and budget, recent alerts and analyses. Each user can reorder and hide sections.
Alerts / Incidents	AI-generated incidents with severity, anomaly type, confidence, evidence and recommended actions; acknowledge; export; per-incident PDF report.
Logs	Advanced log search by severity, server, date range, text or regex, with pagination and export.
Metrics, Services, Usage & Cost	Supporting operational views, each with an honest source label.

Page	Purpose
AI Insights and Urbani Copilot	Incident analyses and the grounded chat assistant, with history, feedback and a quality card.
Service Health	Uptime and status per monitored target over 24 hours, 7 days or 30 days.
Notifications	Channel status, alert rules, "Check connection" and "Send test", and the delivery log.
Audit	Searchable, append-only record of configuration changes and sensitive queries.
Users	Super Admin only: create users, assign roles, activate and deactivate.
Security	Each user manages their own authenticator-app MFA and recovery codes.
Settings	Read-only view of the environment-driven configuration. No secrets are exposed.

3.2 Urbani Copilot (AI assistant)

The Copilot answers operational questions ("Are there any errors right now?", "Summarize the current logs") grounded in the real log window and incident evidence. In live mode the question is sent to the Urbani QA chat endpoint, which grounds the answer server-side on Amazon Bedrock (Nova 2 Lite) across recent log windows and alerts. The assistant is advisory only and never takes action.

- **History:** every exchange is stored per user; users can resume, rename and delete past conversations.
- **Feedback:** users can like or dislike each answer and leave a short comment.
- **Latency:** the end-to-end time of each answer is stored and shown.
- **Quality card:** roles that can view the audit log see satisfaction, counts, latency percentiles and recent dislike comments on the AI Insights page.
- **Privacy:** conversations are owner-only. Another user's session is never readable, not even by a Super Admin. Retention is configurable (placeholder: 90 days).

> **Chat vs. Logs scope**
> The Copilot is grounded server-side across both recent logs and incident alerts, so it may describe an incident that the currently-filtered log window does not list. The assistant surfaces a short grounding note making this scope explicit, so the two views are never mistaken for contradicting each other.

3.3 Log search and export

Logs can be filtered by severity, server, date range and text, or searched with a regular expression. Regular expressions are validated against known catastrophic patterns and run under a strict time budget, so a search can never stall the service. Every search is audited (filters only, never content).

Export	Formats	Notes
Logs	CSV, JSON, PDF	Uses exactly the filters on screen. CSV is Excel-friendly (UTF-8 with BOM) and protected against formula injection.
Alerts	CSV, JSON	Filtered by severity, service, status, date.
Incident report	PDF	One incident with its evidence and recommended actions.

Every export states its source (LIVE, MOCK or WAITING), the scope searched, the filters applied, the row count and whether it was truncated. Limits are configurable (placeholders: 10,000 rows for CSV and JSON, 2,000 rows for PDF, 10 exports per minute per user). Each export is audited. Exported files are not stored.

3.4 Service health and uptime

The Service Health page and a status strip on the Dashboard show UP, DEGRADED or DOWN for each target, with uptime over 24 hours, 7 days or 30 days. Targets are the platform itself, the Urbani telemetry API, and the QA, Main and Payment applications. Targets without a configured health URL show WAITING_FOR_INTEGRATION and are never presented as healthy. Uptime is shown as "insufficient data" until enough samples exist, never as a fake 100%. Probe URLs come only from configuration, and requests to cloud metadata and link-local addresses are always blocked.

3.5 Dashboard personalisation

Each user can reorder and hide Dashboard sections (key metrics, service health, system health, AI cost, observability chart, recent alerts, recent analyses) with drag and drop or keyboard buttons. The layout is saved per user and can be reset to the default. Sections a user is not permitted to see are never shown.

3.6 Branding and dark mode

The interface uses the Urbani logo and colour palette. Users choose System, Light or Dark and the choice is remembered, with no flash on load. Text and interface colours are checked automatically against WCAG AA contrast ratios in both themes.

4. Live Data Integration

The backend integrates with the Urbani QA API Gateway (secured by an x-api-key, called server-side only). Three live data paths are wired through the adapter boundary:

Data	Upstream source	Behaviour
Logs	GET /logs/latest + /logs/history per service	Live log lines, merged latest + history, de-duplicated, newest-first, filterable. Optionally the full CloudWatch Logs Insights stream (LOGS_SOURCE=insights).
Incidents	GET /alerts/latest + /alerts/history	AI-generated incidents; their evidence lines are correlated back to the Logs view.
Chat	POST /chat (Amazon Bedrock Nova 2 Lite)	Server-side grounded answer across recent logs + alerts; raw lines never cross the wire.
Metrics, services detail, usage and cost	Not provisioned upstream	Served as clearly-labelled sample data (MOCK), never claimed LIVE.

4.1 Log evidence correlation

The upstream /logs endpoints currently return only "relevant" (flagged) lines, so the real log content arrives inside each incident's evidence. The backend parses that evidence, preserving original timestamps, inferring levels, and keeping message text verbatim, and surfaces it on the Logs view and in each alert's detail. This keeps the Logs view populated with genuine upstream data and gives every incident verifiable backing evidence. Nothing is ever fabricated.

4.2 Incident confidence

The upstream incident source does not report a model confidence score. Rather than show an empty value, the system derives a confidence from incident severity and labels it clearly as "(derived)" in the UI, with an explanatory tooltip, so it is never mistaken for a model-reported score.

5. Access Control and Security

5.1 Sign-in methods

Method	How it works
Password	Email and password (minimum 10 characters, stored as a bcrypt hash). A signed session token lasts 8 hours by default. Kept as a break-glass login even when Google sign-in is on.
Google single sign-on	Users authenticate at Google; the platform never sees the Google password. Optional, switched on by configuration.

5.2 Google single sign-on

Google proves who the user is. The platform decides whether they may enter and what they may do.

- **Standards:** OpenID Connect authorization-code flow with PKCE. The platform verifies the signature, issuer, audience, expiry and nonce of the Google ID token, and requires a verified e-mail address.
- **Domain restriction:** only e-mail addresses on the allowed domain list may sign in (comma-separated list, exact match).
- **No self-registration:** a user must already exist on the Users page, created by a Super Admin with a role. A Google account that is not on the list is denied, even inside the allowed domain.
- **Roles stay in the platform:** the role a Super Admin chose is used at every sign-in. Deactivating a user blocks their next request.
- **Break-glass administrator:** a local password administrator remains available so the platform can never lock itself out.

5.3 Multi-factor authentication

- **Authenticator app:** time-based one-time codes (TOTP), enrolled from the Security page with a QR code.
- **Recovery codes:** one-time codes for lost devices, which can be regenerated.
- **Policy:** MFA can be made mandatory for chosen roles. Users in a mandatory role are forced to enrol at first sign-in.
- **Protection:** secrets are stored encrypted; repeated wrong codes lock the account for 15 minutes; a Super Admin can reset MFA for a locked-out user (audited).
- **Google users:** multi-factor protection for Google sign-in is enforced by Google (2-Step Verification on the Workspace domain).

5.4 Roles and permissions

The permission matrix below is a DRAFT pending Urbani sign-off. It is defined in one place and read by both the API and the UI.

Permission	Super Admin	Infra Admin	QA Operator	Reader / Auditor
View dashboard, alerts, logs, metrics, services, AI insights, usage, settings	Yes	Yes	Yes	Yes

Permission	Super Admin	Infra Admin	QA Operator	Reader / Auditor
Use Copilot chat (advisory)	Yes	Yes	Yes	Yes
Export logs, alerts, incident report	Yes	Yes	Yes	Yes
Acknowledge alerts, run AI analysis	Yes	Yes	Yes	No
Manage alert rules, update settings	Yes	Yes	No	No
View audit log	Yes	Yes	No	Yes
Manage users	Yes	No	No	No

Enforcement is on the server for every request: the user is loaded on each call, so a role change or deactivation applies immediately. Blocked attempts return 403 and are written to the audit log.

5.5 User management

Super Admins create users on the Users page with an e-mail address, name and role. The password can be left empty for Google-only users. Guard rails: the last active Super Admin cannot be demoted or deactivated, and nobody can change their own role or deactivate themselves.

5.6 Audit log

The audit log is an append-only record with no edit or delete endpoint. It records sign-ins and failures, Google sign-ins and denials, user and role changes, alert acknowledgements, settings changes, searches, exports, chat queries and permission denials. Passwords, tokens, API keys and secrets are redacted before storage, and searches record filters only, never log content. Each event holds the actor, role, IP address, correlation id and result. Retention is configurable (placeholder: 365 days).

5.7 Secrets and data protection

- **Secrets:** the session signing secret, MFA encryption key, Urbani API key, Google client secret and SMTP password are injected from AWS Secrets Manager at deploy time, never baked into images or source.
- **Server-side API key:** the Urbani x-api-key is used only server-side; it is never logged, returned in responses, or sent to the browser.
- **Webhook URLs:** Slack and Teams URLs are secrets and are only accepted for allow-listed hosts.
- **Transport:** TLS terminates at the load balancer; session cookies for Google sign-in are HttpOnly, SameSite and Secure.

5.8 Honest provenance model

Every data surface carries a source badge so viewers always know what they are looking at:

Label	Meaning
LIVE	Real data fetched from the Urbani QA environment.
MOCK	Deterministic sample data for a capability not yet provisioned upstream (for example metrics and usage and cost).
WAITING_FOR_INTEGRATION	A live source is configured but temporarily unavailable, or a target has no endpoint configured. Shown instead of a silent empty state.

No fabrication

The system never invents log lines or AI answers. When a live window is genuinely empty it says so; when an upstream call fails it degrades to an honest "temporarily unavailable" state. This honesty guarantee is enforced throughout the codebase and by tests.

6. Automatic Alert Notifications

The platform notifies the right people automatically when a matching alert appears, so incidents do not wait for someone to open the dashboard.

6.1 How it works

- **Rules:** a rule has a minimum severity, optional services and anomaly types, and the channels to use. A default rule sends Critical and High alerts to all configured channels (DRAFT pending Urbani input).
- **Schedule:** new alerts are checked every 5 minutes and after each live alert refresh.
- **De-duplication and limits:** the same alert is never sent twice on a channel, a per-channel hourly cap prevents floods, and alerts older than the configured age (60 minutes) are not sent.
- **Message:** severity, service, environment, anomaly type, summary, probable cause, confidence (labelled derived), time, and a link that opens the alert in the dashboard.
- **Reliability:** up to three attempts with growing delays; temporary errors are retried, permanent rejections are not.

6.2 Channels

Channel	Delivery	Set-up
E-mail	Google Workspace SMTP (smtp.gmail.com, STARTTLS) or Amazon SNS topic.	Sender account with an app password stored in Secrets Manager. SMTP takes priority over SNS when configured.
Slack	Incoming webhook with formatted blocks and an "Open in dashboard" button.	Webhook URL stored as a secret.
Microsoft Teams	Workflows webhook with an Adaptive Card (legacy connectors also supported).	Webhook URL stored as a secret.

6.3 E-mail recipients by role

E-mail goes to every active user whose role is on the configured list (currently Super Admin, QA Operator and Infrastructure Administrator), read from the Users page at the moment of sending. Adding a user or changing a role takes effect on the next alert with no restart. Fixed extra addresses can also be configured. Addresses on non-deliverable domains are skipped and duplicates are removed.

6.4 Safe rollout and delivery log

- **Dry run:** in dry-run mode nothing is sent; what would have been sent is written to the delivery log so rules and recipients can be reviewed first.
- **Check connection:** for e-mail, signs in to the mail server without sending anything.
- **Send test:** sends a clearly-labelled test message through a channel.
- **Delivery log:** every attempt with its channel, status (sent, failed, dry run, skipped), time and reason, with a configurable retention period.

7. Deployment: AWS ECS Fargate

The application is deployed to ECS Fargate as a single task running two containers (frontend + backend) behind an Application Load Balancer, with TLS terminated at the ALB and the dashboard served at a dedicated domain. Both container images build from the repository root and are pushed to ECR.

7.1 Runtime topology

Setting	Value
Platform	AWS ECS Fargate (serverless containers), awsvpc networking
Task size	0.5 vCPU / 1 GB (512 CPU / 1024 MiB), desired count 1
Frontend container	nginx + React SPA, port 8080 (ALB target)
Backend container	Node/Express, port 4000, reached at 127.0.0.1:4000 within the task
Load balancer	Application Load Balancer, HTTPS :443 (ACM certificate), HTTP to HTTPS redirect
Persistence	SQLite at /data/urbani.sqlite (single writer, so one task). Data survives a redeploy only if /data is persistent storage (see section 10).
Secrets	Session secret, MFA key, Urbani API key, seed admin password, Google client secret and SMTP password from AWS Secrets Manager
Outbound access	Urbani API Gateway, Amazon Bedrock, Google (sign-in) and smtp.gmail.com on port 587 (e-mail)
Logs	CloudWatch Logs with per-container stream prefixes; 30-day retention

7.2 Deploy sequence

- Create ECR repositories and build + push the backend and frontend images from the repo root.
- Register the ECS task definition (two containers, environment and secrets wired, health checks, awslogs).
- Create/attach the ALB target group and listener rules (HTTPS forward + HTTP to HTTPS redirect).
- Create or update the ECS service with deployment circuit-breaker and automatic rollback.
- Point the domain at the ALB via a Route 53 alias record.

7.3 Health checks

- **GET /health:** liveness: always 200 when the process is up (used by the container and ALB health check).
- **GET /ready:** readiness: verifies the database is reachable; returns 503 if not.
- **Frontend /healthz:** nginx liveness target for the container health check.

7.4 Configuration reference

All behaviour is environment-driven. Secrets are referenced by Secrets Manager name only.

Area	Main settings
Core	INTEGRATION_MODE (mock | live), URBANI_API_BASE_URL, URBANI_SERVICES, DB_DRIVER (sqlite | dynamodb), JWT_EXPIRES_IN

Area	Main settings
Sign-in	AUTH_PROVIDER (local | oidc), AUTH_LOCAL_ENABLED, OIDC_ISSUER, OIDC_CLIENT_ID, OIDC_REDIRECT_URI, OIDC_ALLOWED_EMAIL_DOMAIN, OIDC_AUTO_PROVISION, MFA_REQUIRED_ROLES
Notifications	NOTIFICATIONS_ENABLED, NOTIFICATIONS_DRY_RUN, SMTP_HOST, SMTP_PORT, SMTP_USER, SMTP_FROM, ALERT_EMAIL_ROLES, ALERT_EMAIL_TO, DASHBOARD_BASE_URL
Health	HEALTH_TARGETS, HEALTH_PROBES_ENABLED, HEALTH_PROBE_INTERVAL_SECONDS, HEALTH_RETENTION_DAYS
Retention	AUDIT_RETENTION_DAYS, CHAT_RETENTION_DAYS, NOTIFICATIONS_LOG_RETENTION_DAYS
Export	EXPORT_MAX_ROWS, EXPORT_PDF_MAX_ROWS, EXPORT_RATE_LIMIT_PER_MIN, EXPORT_TIMEOUT_MS
Secrets	JWT_SECRET, MFA_ENCRYPTION_KEY, SEED_ADMIN_PASSWORD, URBANI_API_KEY, OIDC_CLIENT_SECRET, SMTP_PASSWORD

8. Operations and Maintenance

- **Zero-downtime deploys:** the ECS service uses a deployment circuit breaker with rollback and keeps healthy capacity during deploys, so a bad image rolls back automatically.
- **Observability:** container logs stream to CloudWatch with separate frontend/backend prefixes; the backend emits structured JSON logs.
- **Configuration:** switching between mock and live data is a single INTEGRATION_MODE setting with no code change.
- **Retention:** daily jobs purge audit events, chat history, notification log and health results after their configured periods.
- **Leavers:** deactivate the user on the Users page (effective on their next request) and also suspend the account in Google Workspace.
- **Secret rotation:** rotate any secret that has been shared outside Secrets Manager; the service reads secrets at start-up, so redeploy after rotation.
- **Rollback:** set AUTH_PROVIDER=local to turn Google sign-in off, clear SMTP_HOST or set NOTIFICATIONS_ENABLED=false to stop e-mail. Stored data is not touched.

> **Security action before go-live**
> The bootstrap admin password and the Urbani API key must be strong values in AWS Secrets Manager (not development defaults), and the previously shared Urbani API key should be rotated. The break-glass administrator should use an address that cannot sign in with Google, and MFA should be required for privileged local logins.

9. Quality and Testing

The backend is covered by an automated test suite that runs as part of the build, with more than 650 tests passing. Coverage includes the live log and alert adapters, evidence parsing and correlation, authentication and MFA, Google sign-in against a simulated identity provider, role enforcement on every route, audit, export, health probing, notification channels and rules, and the chat grounding flow. The full build compiles the shared, backend and frontend packages cleanly, and the frontend passes a strict TypeScript type check.

- **Contract tests:** the same repository tests run against both SQLite and DynamoDB.
- **Honest-data tests:** specific tests assert that empty or absent upstream data never produces fabricated output.
- **Security tests:** role enforcement, secret redaction, regular-expression safety, export formula injection and probe address blocking.
- **Accessibility:** an automatic contrast check of the colour tokens in both themes against WCAG AA, plus keyboard-operable controls. Full accessibility validation still requires manual testing with assistive technologies.
- **Load tests:** k6 scenarios and a Node runner for read, search, export, login and chat traffic. Thresholds are drafts awaiting agreed targets.
- **Deployment checks:** an automatic check that the deployment configuration matches what the backend reads and contains no secret values.

10. Delivery Status and Open Items

This section states plainly what has been exercised and what has not. The running system today uses live Urbani logs, alerts and the Copilot. Items marked pending have been built and tested with simulated services but not yet exercised against the real AWS or third-party service.

10.1 Status by capability

Capability	Built and tested	Pending verification
Live logs, alerts, Copilot	Yes	In production use
Roles, user management, audit	Yes	Permission matrix sign-off by Urbani
Google sign-in	Yes (simulated provider; real Google login confirmed on a local environment)	Production sign-in with the Urbani Workspace domain
Authenticator-app MFA	Yes	MFA policy decision by Urbani
E-mail notifications (SMTP)	Yes (mail server login confirmed locally)	Delivery of a real alert e-mail in production
Slack and Teams notifications	Yes (simulated)	Webhook URLs from Urbani, real test message
Service health panel	Yes (local test servers)	Health URLs for QA, Main and Payment
Log search on CloudWatch Logs Insights	Yes (simulated AWS)	Log group names and read access
Export, personalisation, branding	Yes	Export limits and widget scope confirmed by Urbani

Capability	Built and tested	Pending verification
DynamoDB persistence	Yes (DynamoDB Local)	Table, permissions and approval to deploy
Metrics, services detail, usage and cost	Sample data (labelled MOCK)	Upstream AWS endpoints from Urbani

10.2 Before real users are added

- **Persistent storage:** the database file must live on persistent storage (for example Amazon EFS) or the platform must move to DynamoDB. On ephemeral task storage every redeploy would erase users, audit history and preferences.
- **Single instance:** while SQLite is used, exactly one task runs, so a deployment briefly interrupts service. DynamoDB removes this limit and allows two or more tasks across two availability zones.
- **Secrets:** Google client secret and mail app password created for production and stored only in Secrets Manager; any value shared outside it rotated.
- **Dry run first:** review the notification delivery log in dry-run mode, then switch it off after "Check connection" and "Send test" succeed.

10.3 Inputs requested from Urbani

Item	Needed
Workspace domain(s) allowed to sign in	Domain list
First Super Admin and the policy for creating Super Admins	Google e-mail address and policy
Alert recipients	Confirm the roles that receive e-mail and any fixed addresses
Role permission matrix	Sign-off, including whether auditors may export
MFA policy	Which roles must use MFA
Health check URLs for QA, Main and Payment	URLs and network reachability from the service
Retention and privacy	Audit, chat and notification log periods; chat PII policy
Slack and Teams	Webhook URLs, if those channels are wanted
Metrics, usage and billing	AWS endpoints to replace sample data
Load-test targets	Agreed response-time thresholds and expected traffic
Urbani API key	Confirmation that the previously shared key was rotated

11. Roadmap and Next Steps

Natural extensions beyond the delivered system, each fitting the existing architecture without rework:

- **DynamoDB persistence:** switch the live service to the built DynamoDB driver for durable, multi-task operation.
- **High availability:** run two or more tasks across two availability zones with auto scaling once persistence allows concurrent writers.

- **Live metrics, services and cost:** promote these views from sample data to LIVE when the upstream endpoints exist.
- **Group-based roles:** optionally derive roles from Google Workspace groups instead of manual assignment.
- **Invitation e-mail:** send new users a welcome message with the sign-in link when a Super Admin creates them.
- **Large exports:** stage very large exports in S3 and return a pre-signed link.
- **Native Bedrock analysis:** wire the in-app Bedrock pipeline for on-demand analysis behind the existing provider interface.

12. Glossary

Term	Definition
ALB	Application Load Balancer: the public entry point that terminates TLS and routes to the service.
ECS Fargate	Serverless container runtime; no servers to manage, capacity set as task CPU/ memory.
SSO	Single sign-on: signing in with an existing Google account instead of a separate password.
OIDC / PKCE	OpenID Connect and Proof Key for Code Exchange: the standard, secure protocol used for Google sign-in.
MFA / TOTP	Multi-factor authentication using time-based one-time codes from an authenticator app.
RBAC	Role-based access control: what each user may see and do depends on their role.
Break-glass administrator	A local password account kept for emergencies when Google sign-in is unavailable.
Dry run	A notification mode that records what would be sent without sending anything.
SMTP	The standard protocol for sending e-mail; here used with Google Workspace.
Provenance badge	The LIVE / MOCK / WAITING_FOR_INTEGRATION label shown on every data surface.
Copilot	The in-app AI assistant that answers questions grounded in live logs and incidents.
Incident evidence	The real log lines attached to an AI-generated incident, used as backing proof.
Nova 2 Lite	The Amazon Bedrock model used to generate incident analysis and Copilot answers.
Secrets Manager	AWS service that stores passwords and keys so they never appear in code or images.